Python

1. Python Variables -

declaring variable -

x=3

x=”Sally” or ‘Sally’

x=str(3)

Variables are case sensitives

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Camel Case

Each word, except the first, starts with a capital letter:

myVariableName = "John"

Pascal Case

Each word starts with a capital letter:

MyVariableName = "John"

Snake Case

Each word is separated by an underscore character:

my_variable_name = "John"

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

Example

x, y, z = "Orange", "Banana", "Cherry"

print(x)

print(y)

print(z)

2. Python DataTypes -

Text Type:	`str`
Numeric Types:	`int`, `float`, `complex`
Sequence Types:	`list` [], `tuple` (), `range`
Mapping Type:	`dict`
Set Types:	`set` {}, `frozenset`
Boolean Type:	`bool`
Binary Types:	`bytes`, `bytearray`, `memoryview`
None Type:	`NoneType`

Strings

You can assign a multiline string to a variable by using three quotes:

Example

You can use three double quotes:

a = """Lorem ipsum dolor sit amet,consectetur adipiscing elit,sed do eiusmod tempor incididuntut labore et dolore magna aliqua."""print(a)

```
        Python Data Types
                |
     -------------------------
     |                       |
```

Immutable Mutable
| |

| | | | | | | | | |
int float str tuple ... list dict set bytearray

What Are Strings and What Is String Immutability?

A string is a sequence of characters surrounded by either single or double quotation marks. In some programming languages, characters surrounded by single quotes are treated differently than characters surrounded by double quotes, but in Python, they're treated equally. So, you can use either when working with strings. Here are some examples of strings:

▼ **Example Code**

Example Code

```
my_str_1 = 'Hello'
my_str_2 = "World"
```

If you need a multi-line string, you can use triple double quotes or single quotes:

▼ **Example Code**

Example Code

```
my_str_3 = """Multiline
string"""
my_str_4 = '''Another
multiline
string'''
```

If your string contains either single or double quotation marks, then you have two options:

- Use the opposite kind of quotes. That is, if your string contains single quotes, use double quotes to wrap the string, and vice versa:

▼ **Example Code**

Example Code

```
msg = "It's a sunny day"
quote = 'She said, "Hello World!"'
```

- Escape the single or double quotation mark in the string with a backslash (`\`). With this method, you can use either single or double quotation marks to wrap the string itself:

▼ **Example Code**

Example Code

```
msg = 'It\'s a sunny day'
quote = "She said, \"Hello!\""
```

Sometimes, you may need to check if a string contains one or more characters. For that, Python provides the `in` operator, which returns a boolean that specifies whether the character or characters exist in the string or not.

Here are some examples:

▼ **Example Code**

Example Code

```
my_str = 'Hello world'

print('Hello' in my_str)  # True
print('hey' in my_str)    # False
print('hi' in my_str)    # False
```

```
print('e' in my_str)  # True
print('f' in my_str)  # False
```

Now, let's look at how you can get the length of a string and work with the individual characters in a string, a process called **indexing**. To get the length of a string, you can use the built-in `len()` function. Here's an example:

▼ **Example Code**

Example Code

```
my_str = 'Hello world'
print(len(my_str))  # 11
```

Each character in a string has a position called an index. The index is zero-based, meaning that the index of the first character of a string is `0`, the index of the second character is `1`, and so on. To access a character by its index, you use square brackets (`[]`) with the index of the character you want to access inside. Here are some examples:

▼ **Example Code**

Example Code

```
my_str = "Hello world"

print(my_str[0])  # H
print(my_str[6])  # w
```

Negative indexing is also allowed, so you can get the last character of any string with `-1`, the second-to-last character with `-2`, and so on:

▼ **Example Code**

Example Code

```
my_str = 'Hello world'
print(my_str[-1])  # d
print(my_str[-2]) # l
```

Many other programming languages group data types broadly as either primitive or reference types. Primitive types are simple and immutable, meaning they can't be changed once declared. Reference types can hold multiple values, and are either mutable or immutable. But Python doesn't draw a hard line between those

two groups. Instead, all data gets treated as objects, and some objects are immutable while others are mutable.

Immutable data types can't be modified or altered once they're declared. You can point their variables at something new, which is called reassignment, but you can't change the original object itself by adding, removing, or replacing any of its elements.

Strings are immutable data types in Python. This means that you can reassign a different string to a variable:

▼ **Example Code**

Example Code

```
greeting = 'hi'
greeting = 'hello'
print(greeting) # hello
```

But direct modification of a string isn't allowed:

▼ **Example Code**

Example Code

```
greeting = 'hi'
greeting[0] = 'H' # TypeError: 'str' object does not support item assignment
```

Examples of other immutable data types in Python are integer, float, boolean, tuple, and range. You'll get to know each of these types in upcoming lessons

What Are String Concatenation and String Interpolation?

When working with strings, combining different pieces of text together is a common operation you'll often find yourself dealing with.

In Python, you can combine multiple strings together with the plus (`+`) operator. This process is called **string concatenation**. Here's how to concatenate two strings with the plus operator:

▼ **Example Code**

Example Code

```
my_str_1 = 'Hello'
my_str_2 = "World"

str_plus_str = my_str_1 + ' ' + my_str_2
print(str_plus_str) # Hello World
```

But note that this only works with strings. If you try to concatenate a string with a number, you'll get a `TypeError`:

▼ **Example Code**

Example Code

```
name = 'John Doe'
age = 26

name_and_age = name + age
print(name_and_age) # TypeError: can only concatenate str (not "int") to str
```

This happens because Python does not automatically convert other data types like integers into strings when you concatenate them. Python requires all elements to be strings before it can concatenate them. To fix that, you can convert the number into a string with the built-in `str()` function, which returns the string representation of the given object without modifying the original object:

▼ **Example Code**

Example Code

```
name = 'John Doe'
age = 26

name_and_age = name + str(age)
print(name_and_age) # John Doe26
```

You can also use the augmented assignment operator for concatenation. This is represented by a plus and equals sign (`+=`), and performs both concatenation and assignment in one step. Here's it in action:

▼ **Example Code**

Example Code

```
name = 'John Doe'
age = 26

name_and_age = name  # Start with the name
```

```
name_and_age += str(age)  # Append the age as string

print(name_and_age)  # John Doe26
```

The process of inserting variables and expressions into a string is called **string interpolation**. Python has a category of string called **f-strings** (short for formatted string literals), which allows you to handle interpolation with a compact and readable syntax.

F-strings start with `f` (either lowercase or uppercase) before the quotes, and allow you to embed variables or expressions inside replacement fields indicated by curly braces (`{}`). Here's an example:

▼ **Example Code**

Example Code

```
name = 'John Doe'
age = 26
name_and_age = f'My name is {name} and I am {age} years old'
print(name_and_age) # My name is John Doe and I am 26 years old

num1 = 5
num2 = 10
print(f'The sum of {num1} and {num2} is {num1 + num2}') # The sum of 5 and 10 is 15
```

Note how you don't need to convert non-string types with the `str()` function. In the example above, the value of the `age`, `num1`, and `num2` variables is converted under the hood into a string during the interpolation process.

String interpolation is the process of inserting variables or expressions directly into a string.

What Is String Slicing and How Does It Work?
In a previous lesson, you learned how each character in a string can be identified by its index (starting from zero), and accessed using the bracket notation:**Example Code**

```
my_str = "Hello world"

print(my_str[0])  # H
print(my_str[6])  # w
print(my_str[-1]) # d
```

String slicing lets you extract a portion of a string or work with only a specific part of it. Here's the basic syntax:**Example Code**

```
string[start:stop]
```

If you want to extract characters from a certain index to another, you just separate the `start` and `stop` indices with a colon:**Example Code**

```
my_str = 'Hello world'
print(my_str[1:4]) # ell
```

Note that the `stop` index is non-inclusive, so `[1:4]` just extracted the characters from index `1`, and up to, but not including, the character at index `4`.
You can also omit the `start` and `stop` indices, and Python will default to `0` or the end of the string, respectively. For example, here's what happens if you omit the `start` index:**Example Code**

```
my_str = 'Hello world'
print(my_str[:7])  # Hello w
```

This extracts everything from index `0` up to (but not including), the character at index `7`. And here's what happens if you omit the `stop` index:**Example Code**

```
my_str = 'Hello world'
print(my_str[8:])  # rld
```

This extracts everything from the character at index `8` until the end of the string. Note that slicing a string does not modify the original string:**Example Code**

```
my_str = 'Hello world'
print(my_str[8:])  # rld
print(my_str)  # Hello world
```

You can also omit both the `start` and `stop` indices, which will extract the whole string:**Example Code**

```
my_str = 'Hello world'
print(my_str[:])  # Hello world
```

Apart from the `start` and `stop` indices, there's also an optional `step` parameter, which is used to specify the increment between each index in the slice.
Here's the syntax for that:**Example Code**

```
string[start:stop:step]
```

In the example below, the slicing starts at index `0`, stops before `11`, and extracts every second character:**Example Code**

```
my_str = 'Hello world'
print(my_str[0:11:2])  # Hlowrd
```

A helpful trick you can do with the `step` parameter is to reverse a string by setting step to `-1`, and leaving `start` and `stop` blank:**Example Code**

```
my_str = 'Hello world'
print(my_str[::-1]) # dlrow olleH
```

QuestionsHow do you extract a specific portion of a string in Python?Using parentheses with start and end positions.Using curly braces with start and stop positions.Using square brackets with start and stop positions.Using angle brackets with start and stop positions.What is the outcome of `'Hello'[2:]`?
`llloloell`What is the purpose of the optional `step` parameter in string slicing?It specifies if the sliced string should be changed in place or not.It specifies the increment between each character to extract.It ensures that all characters are extracted.It specifies a maximum number of repeated characters to extract.Check your answerAsk for Help

What Are Some Common String Methods?

Python provides a number of built-in methods that make working with strings a breeze. They include, but are not limited to, the following:

- `upper()`: Returns a new string with all characters converted to uppercase.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

uppercase_my_str = my_str.upper()
print(uppercase_my_str)  # HELLO WORLD
```

- `lower()`: Returns a new string with all characters converted to lowercase.

▼ **Example Code**

Example Code

```
my_str = 'Hello World'

lowercase_my_str = my_str.lower()
print(lowercase_my_str)  # hello world
```

- `strip()`: Returns a new string with the specified leading and trailing characters removed. If no argument is passed it removes leading and trailing whitespace.

▼ **Example Code**

Example Code

```
my_str = '  hello world  '

trimmed_my_str = my_str.strip()
print(trimmed_my_str)  # "hello world"
```

- `replace(old, new)`: Returns a new string with all occurrences of `old` replaced by `new`.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

replaced_my_str = my_str.replace('hello', 'hi')
print(replaced_my_str)  # hi world
```

- `split(separator)`: Splits a string on a specified separator into a list of strings. If no separator is specified, it splits on whitespace.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

split_words = my_str.split()
print(split_words)  # ['hello', 'world']
```

- `join(iterable)`: Joins elements of an iterable into a string with a separator.

▼ **Example Code**

Example Code

```
my_list = ['hello', 'world']

joined_my_str = ' '.join(my_list)
print(joined_my_str)  # hello world
```

- `startswith(prefix)`: Returns a boolean indicating if a string starts with the specified prefix.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

starts_with_hello = my_str.startswith('hello')
print(starts_with_hello)  # True
```

- `endswith(suffix)`: Returns a boolean indicating if a string ends with the specified suffix.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

ends_with_world = my_str.endswith('world')
print(ends_with_world)  # True
```

- `find(substring)`: Returns the index of the first occurrence of `substring`, or `1` if it doesn't find one.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

world_index = my_str.find('world')
print(world_index)  # 6
```

- `count(substring)`: Returns the number of times a substring appears in a string.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

o_count = my_str.count('o')
print(o_count)  # 2
```

- `capitalize()`: Returns a new string with the first character capitalized and the other characters lowercased.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

capitalized_my_str = my_str.capitalize()
print(capitalized_my_str)  # Hello world
```

- `isupper()`: Returns `True` if all letters in the string are uppercase and `False` if not.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

is_all_upper = my_str.isupper()
print(is_all_upper)  # False
```

- `islower()`: Returns `True` if all letters in the string are lowercase and `False` if not.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

is_all_lower = my_str.islower()
print(is_all_lower)  # True
```

- `title()`: Returns a new string with the first letter of each word capitalized.

▼ **Example Code**

Example Code

```
my_str = 'hello world'

title_case_my_str = my_str.title()
print(title_case_my_str)  # Hello World
```

2. Lists

a=[1,2,3]

a[0]=1

a[1]=2

a[3]=3

a[-1]=3

len(a)

del a[0]

```
developer = ['Alice', 25, ['Python', 'Rust', 'C++']]
developer[2][1] # 'Rust'
```

```
developer = ['Alice', 34, 'Rust Developer']
name, age, job = developer

print(name) # 'Alice'
print(age) # 34
print(job) # 'Rust Developer'
```

```
developer = ['Alice', 34, 'Rust Developer']
name, *rest = developer

print(name) # 'Alice'
print(rest) # [34, 'Rust Developer']
```

Method used in list

1. append()- add item to the end of the list

```
numbers = [1, 2, 3, 4, 5]
numbers.append(6)
print(numbers) # [1, 2, 3, 4, 5, 6]
```

```
numbers = [1, 2, 3, 4, 5]
even_numbers = [6, 8, 10]

numbers.append(even_numbers)
print(numbers) # [1, 2, 3, 4, 5, [6, 8, 10]]
```

2. extend()

```
numbers = [1, 2, 3, 4, 5]
even_numbers = [6, 8, 10]

numbers.extend(even_numbers)
print(numbers) # [1, 2, 3, 4, 5, 6, 8, 10]
```

3. insert() - specific index in list

```
numbers = [1, 2, 3, 4, 5]
numbers.insert(2, 2.5)

print(numbers) # [1, 2, 2.5, 3, 4, 5]
```

4. remove() - first occurrence of an item

```
numbers = [10, 20, 30, 40, 50, 50]
numbers.remove(50)

print(numbers) # [10, 20, 30, 40, 50]
```

5. pop()-to remove specific element if not specified then last element

```
numbers = [1, 2, 3, 4, 5]
numbers.pop(1)
```

6. clear() - to remove everything from list

```
numbers = [1, 2, 3, 4, 5]
numbers.clear()

print(numbers) # []
```

7. sort -

```
numbers = [19, 2, 35, 1, 67, 41]
numbers.sort()

print(numbers) # [1, 2, 19, 35, 41, 67]
```

In contrast to the `sort()` method, there is the `sorted()` function which works for any iterable and returns a new sorted list instead of modifying the original list. For example:

```
numbers = [19, 2, 35, 1, 67, 41]
sorted_numbers = sorted(numbers)

print(numbers) # [19, 2, 35, 1, 67, 41]
print(sorted_numbers) # [1, 2, 19, 35, 41, 67]
```

3. Tuple

```
developer = ('Alice', 34, 'Rust Developer')
developer[1] # 34
```

Methods different from List

1. count()

```
programming_languages = ('Rust', 'Java', 'Python', 'C++', 'Ru
st')
programming_languages.count('Rust') # 2
```

Zip Function

```
developers = ['Naomi', 'Dario', 'Jessica', 'Tom']
ids = [1, 2, 3, 4]

list(zip(developers, ids))
# [('Naomi', 1), ('Dario', 2), ('Jessica', 3), ('Tom', 4)]
```

Filter Function

```
words = ['tree', 'sky', 'mountain', 'river', 'cloud', 'sun']

def is_long_word(word):
    return len(word) > 4

long_words = list(filter(is_long_word, words))
print(long_words) # ['mountain', 'river', 'cloud']
```

Map Function

Error Handling

OOPS

Socket Programming -

File Handling